NLP · Day 27 of 30 · about 60 minutes

Text basics: cleaning and tokenising

🎯 Today's goal

By the end of this hour you will:

- know why text must be **cleaned** before a computer can analyse it
- lowercase text and remove punctuation
- split text into words, called **tokens**, in three different ways
- remove **stop words** and shorten words to their **stem**
- count the most common words and build your own `clean_text()` function

🤔 Why this matters

Welcome to the last phase of PyPath: **NLP**, short for *Natural Language Processing*. "Natural language" means the language people write and speak, like English. NLP is how computers work with text: spam filters, search engines, translation apps, chatbots and the tools that read thousands of product reviews to tell a shop what customers think.

So far all our data was numbers in neat tables. Text is messier. To a computer, `"Great"`, `"great"` and `"great!!!"` are three completely different things. Cleaning text is like washing vegetables before cooking: not exciting, but every recipe starts with it. Over the next four days you'll go from raw text to a program that reads reviews and decides whether they are happy or unhappy.

💬 Our data: customer reviews

Here are 12 **made-up** reviews of made-up wireless headphones from a made-up online shop. In NLP, one piece of text (a review, an email, a tweet) is called a **document**, and a collection of documents is a **corpus**. Our corpus is simply a Python list of strings:

```
# 12 made-up reviews (invented for this lesson)
reviews = [
    "Great headphones! The sound is AMAZING and the battery lasts all day.",
    "Terrible. The battery died after two days... very disappointed.",
    "I love the colour, but the case feels cheap.",
    "Fast delivery, great price, great sound!!!",
    "Not worth the money. The sound is flat and the strap broke.",
    "Comfortable to wear for hours. I'd buy them again.",
    "The app keeps crashing; the headphones themselves are fine.",
    "Loved it! Best gift I've bought this year.",
    "Battery life is poor and charging is slow.",
    "Good sound for the price, but the bass is weak.",
    "Broke after one week. Customer service never replied!",
    "Lovely design, loving the sound, love the battery. 10/10",
]

print(len(reviews), "reviews")
print(reviews[0])
```

```
12 reviews
Great headphones! The sound is AMAZING and the battery lasts all day.
```

🔡 Step 1: lowercase everything

Python compares text letter by letter, and capital letters count. For counting words we want `"Great"` and `"great"` to be the same word, so the first cleaning step is `.lower()`, which you met on Day 2:

```
print("Great" == "great")       # capitals make them different
print("great!!!" == "great")    # so does punctuation

first = reviews[0].lower()
print(first)
```

```
False
False
great headphones! the sound is amazing and the battery lasts all day.
```

✂️ Step 2: remove punctuation

Python's built-in `string` module has a ready-made text called `string.punctuation` holding every punctuation mark. Together with two string tools we can delete them all at once:

- `str.maketrans("", "", string.punctuation)` builds a **translation table**: a set of instructions saying "replace nothing with nothing, and delete these characters".
- `text.translate(table)` follows those instructions on a piece of text.

```
import string

print(string.punctuation)

remove_punct = str.maketrans("", "", string.punctuation)
no_punct = first.translate(remove_punct)
print(no_punct)
```

```
!"#$%&'()*+,-./:;<=>?@[\]^_`{|}~
great headphones the sound is amazing and the battery lasts all day
```

🧩 Step 3: split into tokens

A **token** is one small piece of text, usually one word. Splitting text into tokens is called **tokenising**. The simplest way is `.split()`, which cuts the text wherever there are spaces and gives back a list:

```
tokens = no_punct.split()
print(tokens)
print(len(tokens), "tokens")
```

```
['great', 'headphones', 'the', 'sound', 'is', 'amazing', 'and', 'the',
'battery', 'lasts', 'all', 'day']
12 tokens
```

Removing *all* punctuation has a downside: the apostrophe in `i'd` disappears and you get `id`, which is a different word. A more careful tool is a **regular expression** (from the `re` module): a tiny pattern language for finding text. `re.findall(pattern, text)` returns every piece of the text that matches the pattern.

Pattern piece	Means
`[a-z']`	one character that is a letter from a to z, or an apostrophe
`+`	"one or more of the thing before me, in a row"
`[a-z']+`	a run of letters (and apostrophes): a word!
`[^a-z' ]`	the `^` means NOT: any character that is not a letter, apostrophe or space

The `r` before the quotes makes a **raw string**, which keeps backslashes exactly as typed. Patterns often contain backslashes, so programmers always write patterns as raw strings.

```
import re

review = reviews[5].lower()
print(review)

print(review.translate(remove_punct).split())   # "i'd" becomes "id"
print(re.findall(r"[a-z']+", review))           # "i'd" stays "i'd"

# re.sub replaces every match: here, anything that isn't a letter, ' or space
print(re.sub(r"[^a-z' ]", "", review))
```

```
comfortable to wear for hours. i'd buy them again.
['comfortable', 'to', 'wear', 'for', 'hours', 'id', 'buy', 'them', 'again']
['comfortable', 'to', 'wear', 'for', 'hours', "i'd", 'buy', 'them', 'again']
comfortable to wear for hours i'd buy them again
```

A third option comes from **NLTK** (the Natural Language Toolkit), a big Python library for NLP. Its `wordpunct_tokenize` keeps punctuation, but as separate tokens. That's useful when punctuation carries meaning, like `!!!`:

```
from nltk.tokenize import wordpunct_tokenize

print(wordpunct_tokenize("Loved it! I'd buy again... 10/10"))
```

```
['Loved', 'it', '!', 'I', "'", 'd', 'buy', 'again', '...', '10', '/', '10']
```

Good to know: NLTK's most famous tokeniser, `word_tokenize`, needs extra data files that must be downloaded. This page can't download them, so we use `re.findall` and `wordpunct_tokenize`, which need no downloads. On your own computer, `nltk.download()` gets the extra files.

📚 Cleaning every review

So far we cleaned one review. A natural first try is to lowercase the whole list at once. Read the error carefully:

💡 This example has a mistake on purpose. Run it and read the error.

```
reviews.lower()
```

> ⚠️ **AttributeError**
>
> This kind of value doesn't have that. Check the spelling after the dot.
>
> ```
> Traceback (most recent call last):
>   line 1, in <module>
>     reviews.lower()
>     ^^^^^^^^^^^^^
> AttributeError: 'list' object has no attribute 'lower'
> ```

`'list' object has no attribute 'lower'`: `.lower()` belongs to *strings*, and `reviews` is a *list* of strings. The fix is a loop (or a list comprehension from Day 5) that cleans each string one at a time:

```
lower_reviews = [r.lower() for r in reviews]

for r in lower_reviews[:3]:      # show the first three
    print(r)
```

```
great headphones! the sound is amazing and the battery lasts all day.
terrible. the battery died after two days... very disappointed.
i love the colour, but the case feels cheap.
```

🔢 Counting words

Which words do customers use most? Python's `collections` module has a `Counter`: give it a list, and it counts how often each item appears (like the counters you built with loops on Day 5, but for every item at once). Its `.most_common(n)` method gives the top *n* items as `(word, count)` pairs:

```
from collections import Counter

all_words = []
for review in reviews:
    all_words += re.findall(r"[a-z']+", review.lower())

counts = Counter(all_words)
print(len(all_words), "words in total,", len(counts), "different words")
counts.most_common(8)
```

```
108 words in total, 73 different words
[('the', 14), ('sound', 5), ('is', 5), ('battery', 4), ('great', 3), ('and',
3), ('headphones', 2), ('after', 2)]
```

The winner is `the`, and `is` and `and` are near the top. True, but useless: these words say nothing about headphones.

🛑 Step 4: remove stop words

Stop words are very common words that carry little meaning on their own: *the, a, is, and, it, to*… Removing them leaves the words that matter. NLTK has a ready-made list, but it is another download, so we write our own short list.

We store it as a **set**: curly brackets `{ }` with values but no `key: value` pairs. A set is like a list without order or repeats, and checking `word in stop_words` is very fast.

```
stop_words = {
    "a", "an", "the", "and", "or", "but", "is", "are", "was", "it", "it's",
    "i", "i'd", "i've", "this", "them", "to", "of", "for", "all", "after",
    "very", "feels", "keeps",
}

words = re.findall(r"[a-z']+", reviews[0].lower())
kept = [w for w in words if w not in stop_words]

print(words)
print(kept)
```

```
['great', 'headphones', 'the', 'sound', 'is', 'amazing', 'and', 'the',
'battery', 'lasts', 'all', 'day']
['great', 'headphones', 'sound', 'amazing', 'battery', 'lasts', 'day']
```

Careful with "not": many stop-word lists include `not` and `never`. But *"not worth the money"* means the opposite of *"worth the money"*! Our list keeps them on purpose. Which words to drop depends on your task; you'll see this problem again on Day 29.

🌱 Step 5: stemming

love, *loved*, *loving* and *lovely* are four tokens, but they share one idea. **Stemming** chops word endings to get a common root, called the **stem**. NLTK's `PorterStemmer` follows a famous set of rules from 1980. Stems don't have to be real words (`batteri`), they only have to match:

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()
for word in ["love", "loved", "loving", "lovely", "battery", "batteries", "charging",
"amazing"]:
    print(f"{word:>10} -> {stemmer.stem(word)}")
```

```
      love -> love
     loved -> love
    loving -> love
    lovely -> love
   battery -> batteri
 batteries -> batteri
  charging -> charg
   amazing -> amaz
```

🧼 Putting it together: `clean_text()`

A series of steps where the output of one is the input of the next is called a **pipeline**. Let's wrap all our steps in one function (Day 6), so any text can be cleaned with a single call:

```
def clean_text(text):
    """Lowercase, find the words, drop stop words, stem. Returns a list of tokens."""
    words = re.findall(r"[a-z']+", text.lower())
    words = [w for w in words if w not in stop_words]
    return [stemmer.stem(w) for w in words]

print(reviews[11])
print(clean_text(reviews[11]))
```

```
Lovely design, loving the sound, love the battery. 10/10
['love', 'design', 'love', 'sound', 'love', 'batteri']
```

Now count again, using the cleaned tokens of every review. A Counter's `.update(list)` adds more items to the count:

```
clean_counts = Counter()
for review in reviews:
    clean_counts.update(clean_text(review))

clean_counts.most_common(8)
```

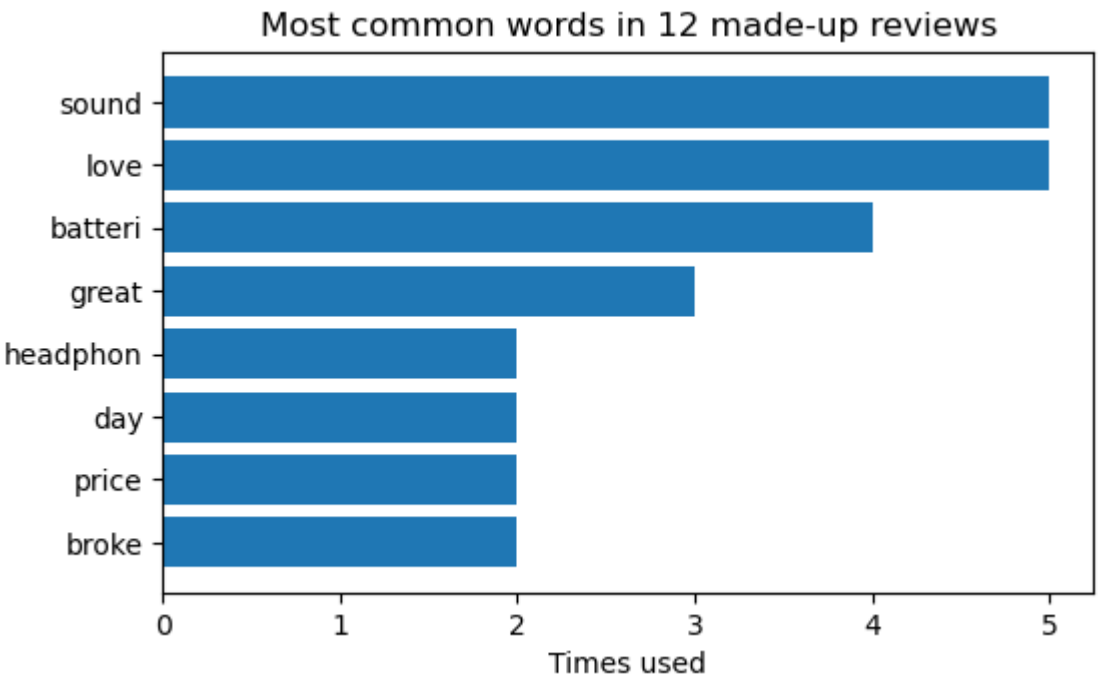
```
[('sound', 5), ('love', 5), ('batteri', 4), ('great', 3), ('headphon', 2),
('day', 2), ('price', 2), ('broke', 2)]
```

Much more useful! Customers talk about the **sound**, the **battery** and the **price**, and `love` now collects *love, loved, loving* and *lovely* together. A quick chart (Day 13) makes it clear:

```
import matplotlib.pyplot as plt

top = clean_counts.most_common(8)
words = [word for word, count in top]
numbers = [count for word, count in top]

plt.figure(figsize=(6, 3.5))
plt.barh(words[::-1], numbers[::-1], color="tab:blue")   # [::-1] puts the biggest on
top
plt.title("Most common words in 12 made-up reviews")
plt.xlabel("Times used")
plt.show()
```



⚠️ Common mistakes

- Calling a string method on a list: `reviews.lower()` → `AttributeError`. Loop over the list instead.
- Forgetting to lowercase, so `Great` and `great` are counted separately.
- Removing every punctuation mark, which turns `i'd` into `id`. Choose your cleaning on purpose.
- Dropping `not` as a stop word when the meaning matters.
- Expecting stems to be real words. `batteri` is fine: it only has to match.

✅ Recap

- A **document** is one text; a **corpus** is a collection of them.
- Cleaning: `.lower()`, then remove punctuation with `str.maketrans` + `.translate` or `re.sub`.
- **Tokens** are the pieces of text: `.split()`, `re.findall(r"[a-z']+", text)` or NLTK's `wordpunct_tokenize`.
- **Stop words** are common words with little meaning; filter them with a set.
- **Stemming** (`PorterStemmer`) cuts words to a shared root.
- `Counter(list).most_common(n)` finds the most frequent words.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

1. Tidy a shouting review

Make the review lowercase and remove all punctuation. Store the result in a variable called `clean`. It should become `wow best coffee maker ever`.

```
import string

review = "WOW!!! Best Coffee Maker, EVER."

# Store the cleaned text in a variable called clean
```

Hints:

1. Start with `review.lower()`.
2. Make a table: `table = str.maketrans("", "", string.punctuation)`
3. `clean = review.lower().translate(table)`

2. Split into tokens

Turn the sentence into a list of words without any punctuation, and store it in a variable called `tokens`. Use `re.findall` (or clean first and then `.split()`).

```
import re

sentence = "the sound is great, the price is great too!"

# Store the list of words in a variable called tokens
```

Hints:

1. The pattern for a word is `r"[a-z']+"`.
2. `tokens = re.findall(r"[a-z']+", sentence)`

3. Remove the stop words

Make a new list called `kept` with only the tokens that are **not** in `stop_words`, in the same order.

```
tokens = ["the", "battery", "is", "not", "great", "and", "the", "case", "is",
"cheap"]
stop_words = {"the", "is", "and", "a"}

# Store the tokens that are not stop words in a list called kept
```

Hints:

1. Loop over `tokens` and keep a word only `if word not in stop_words`.
2. As a list comprehension: `[w for w in tokens if ...]`
3. `kept = [w for w in tokens if w not in stop_words]`

4. The most common word

Count every word in these five tiny reviews with a `Counter`, stored in `word_counts`. Then store the most common word in `top_word` and how many times it appears in `top_count`.

```
from collections import Counter

reviews = ["great sound great price", "poor sound", "great battery", "battery
died", "great case"]

# Count all the words in word_counts, then find top_word and top_count
```

Hints:

1. Collect every word first: start with `all_words = []` and, for each review, add `review.split()`.
2. `word_counts = Counter(all_words)`. Then `word_counts.most_common(1)` gives a list with one `(word, count)` pair.
3. `top_word, top_count = word_counts.most_common(1)[0]`

5. Challenge: your own `clean_text()`

Finish the function `clean_text(text)` so it returns a list of cleaned tokens:

1. lowercase the text,
2. find the words with `re.findall(r"[a-z']+", ...)`,
3. drop the words in `stop_words`,
4. stem each remaining word with `stemmer.stem()`.

For example `clean_text("The batteries were AMAZING!!!")` should return `['batteri', 'were', 'amaz']`. The checker will try it on several texts.

```
import re
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()
stop_words = {"the", "a", "an", "and", "is", "it", "i", "this", "my", "too"}

def clean_text(text):
    # 1. lowercase  2. find the words  3. drop stop words  4. stem each word
    return []

print(clean_text("The batteries were AMAZING!!!"))
```

Hints:

1. Step 2: `words = re.findall(r"[a-z']+", text.lower())`
2. Step 3: `words = [w for w in words if w not in stop_words]`
3. Step 4: `return [stemmer.stem(w) for w in words]`

➡️ Tomorrow

Bag-of-words and TF-IDF: machine learning models only understand numbers. Tomorrow you'll turn each document into a row of numbers, let scikit-learn build the vocabulary for you, find out which words really matter in each text, and build a tiny search engine that finds the document most similar to a question.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. Tidy a shouting review

```
import string

review = "WOW!!! Best Coffee Maker, EVER."

table = str.maketrans("", "", string.punctuation)
clean = review.lower().translate(table)
print(clean)
```

2. Split into tokens

```
import re

sentence = "the sound is great, the price is great too!"

tokens = re.findall(r"[a-z']+", sentence)
print(tokens)
```

3. Remove the stop words

```
tokens = ["the", "battery", "is", "not", "great", "and", "the", "case", "is",
"cheap"]
stop_words = {"the", "is", "and", "a"}

kept = [w for w in tokens if w not in stop_words]
print(kept)
```

4. The most common word

```
from collections import Counter

reviews = ["great sound great price", "poor sound", "great battery", "battery
died", "great case"]

all_words = []
for review in reviews:
    all_words += review.split()

word_counts = Counter(all_words)
top_word, top_count = word_counts.most_common(1)[0]
print(top_word, top_count)
```

5. Challenge: your own clean_text()

```
import re
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()
stop_words = {"the", "a", "an", "and", "is", "it", "i", "this", "my", "too"}

def clean_text(text):
    words = re.findall(r"[a-z']+", text.lower())
    words = [w for w in words if w not in stop_words]
    return [stemmer.stem(w) for w in words]

print(clean_text("The batteries were AMAZING!!!"))
```